Application Discovery and Management Core

1. Course Objectives

After the GUI system starts, where do the desktop icons come from? Why can the system recognize it when we put a folder in the SD card? In this lesson, we will introduce the "application discovery" mechanism of `AppManager`.

2. Core Source Code Analysis

File Path: `src/canmv/port/builtin_py/ybMain/main.py`

The `scan_apps` method in `AppManager` (around line 689) is responsible for scanning and loading applications. This method demonstrates how to achieve pluginization through Python's dynamic features.

2.1 Application Storage Structure

All applications are stored in the `/sdcard/apps/` directory. This is a typical **plugin-based** structure.

```
/sdcard/apps/
├── camera/           <-- This is an App
|   ├── app.py        <-- Code entry point
|   └── icon.png      <-- Desktop icon
├── setting/          <-- This is another App
|   ├── app.py
|   └── icon.png
└── ...
```

2.2 Dynamic Loading Principle

The loading process of standard plugins (apps) is as follows:

1. **Traverse Directory**: When the system starts, `AppManager` will list all subdirectories under the `apps` directory.
2. **Verify Legitimacy**: Check if there is `app.py` in the folder.
3. **Import Module**: Use Python's `import` mechanism to dynamically load `app.py`.
4. **Instantiate**: Each `app.py` contains an `App` class, the system will create its instance and add it to the management list.

2.3 Application Registration and Icon Creation (`register_app` & `create_app_icon`)

After the application is instantiated, it will be added to the `self.apps` dictionary. Subsequently, `AppManager` will call `create_app_icon` to draw desktop icons.
This function is not just about drawing an image, it also contains exquisite animation logic:

1. **Color Generation**:

- If the App comes with its own `icon`, use the image.
- If there is no image but `color` is defined, automatically generate a gradient background.
- If there is nothing, select colors from a preset iOS-style palette in rotation.

2. **Entrance Animation**:
   - We use `lv.anim_t` to create three concurrent animations: icon grows from small to large (`size_anim`), icon fades in (`opa_anim`), text fades in (`label_anim`).
   - **Wave Effect**: `delay_ms = (row * self.icons_per_row + col) * 20`. By calculating delay time based on row and column positions, icons pop up one by one like waves.

2.4 Implementation of Page Turning Logic (`change_page`)

When the number of applications exceeds one page, or when the user swipes the screen, page turning is needed. The implementation logic of page turning is very concise and effective:

1. **Clear Old Page**:
   - Traverse all child objects on the main screen (`self.home_screen`).
   - Determine whether the child object is within the "application grid area" through coordinates.
   - If it's an application icon (not status bar or Dock), directly `delete()` it.

```
# Pseudocode logic
for child in home_screen.children:
    if child.y >= grid_start_y:
        child.delete()
```

2. **Create New Page**:
   - Update `self.current_page` variable.
   - Re-traverse all Apps, calculate whether they should be displayed on the new page.
   - If they should be displayed, call `create_app_icon` again, which means each page turn will trigger a cool icon popup animation.

```
app_page = (app_index) // self.apps_per_page + 1
if app_page == page_number:
    self.create_app_icon(self.apps[app_name])
```